

1 MDS-MleOutboundUtil

1.1 Purpose

The library MleOutboundUtil provides helper functions that you can use to process the data of the MLE outbound transactions.

1.2 Requirements

To use the library MleOutboundUtil, you require SP16.

1.3 Code example flat output data

This code example shows an ExternalJavaService that processes the MLE output data.

The service reads all segments of type "RESTOUTBOUND1LEVEL". Then the service uses the order and the operation data from the MLE output data to create orders and operations via WSP and REST call.

```
package de.mpdv.extSvc.mleoutboundprocessor;

import static de.mpdv.commonmleoutboundutil.MleOutboundUtil.*;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.databind.ObjectMapper;
import de.mpdv.commonmleoutboundutil.BeginOutboundTaParam;
import de.mpdv.commonmleoutboundutil.ControlRecordData;
import de.mpdv.commonmleoutboundutil.EndOutboundTaParam;
import de.mpdv.commonmleoutboundutil.MleFileData;
import de.mpdv.commonmleoutboundutil.MleOutboundException;
import de.mpdv.commonmleoutboundutil.ProcessingState;
import de.mpdv.commonmleoutboundutil.ProtocolFileData;
import de.mpdv.commonmleoutboundutil.RecordData;
import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.sdi.data.SdiException;
import de.mpdv.sdi.data.SesContext;
import de.mpdv.sdi.data.SesRequest;
import de.mpdv.sdi.data.SesResult;
import de.mpdv.sdi.simpleExternalService.ISimpleExternalService;
import de.mpdv.sdi.systemutility.IDbConnectionProvider;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.OutputStream;
import java.net.CookieManager;
import java.nio.charset.StandardCharsets;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.Arrays;
import java.util.Calendar;
import java.util.Collections;
import java.util.List;
import java.util.function.ToIntBiFunction;

public class MleOutboundProcessorService implements ISimpleExternalService
{
    @Override
    public SesResult execute(final SesRequest request,
        final SesContext context,
        final ISystemUtilFactory factory)
    {
        final String segmentName = "RESTOUTBOUND1LEVEL";

        final Calendar now = (Calendar) context.getHydraNow().clone();
        final IDbConnectionProvider dbConnectionProvider = factory.fetchUtil("DbConnectionProvider");
        try (final Connection con = dbConnectionProvider.fetchDbConnection())
        {
            ProcessingState state = ProcessingState.DONE;
            Throwable error = null;

            final String processIdentifier = createUniqueIdentifier();
            final List<RecordData> recordDataList = beginOutboundTa(new BeginOutboundTaParam(con, processIdentifier, now,
segmentName));
            if (recordDataList.isEmpty())
            {

```

```

        return null;
    }
    try
    {
        callServices(recordDataList);
    } catch (final IOException exc)
    {
        error = exc;
        state = ProcessingState.ERROR;
        final String msg = "Error calling services";
        throw new SdiException("InvalidState", msg, exc, msg);
    } catch (final RuntimeException | Error exc)
    {
        error = exc;
        state = ProcessingState.ERROR;
        throw exc;
    } finally
    {
        final MleFileData protFileData = writeProtocolFile(writeFile(factory), "Protocol data", now);
        final MleFileData errFileData = writeErrorFile(writeFile(factory), "Error data", now);
        final MleFileData dataFileData = writeDataFile(writeFile(factory), "Data", now);
        endOutboundTa(new EndOutboundTaParam(con, processIdentifier,
            Collections.singletonList(
                new ControlRecordData(createUniqueIdentifier(), state)
                    .withProtocolFileDataList(Collections.singletonList(
                        new ProtocolFileData("Test", "TstPrg", protFileData.getFileName(), protFileData.getFileSize(),
                            errFileData.getFileName(), errFileData.getFileSize(), dataFileData.getFileName(),
                                dataFileData.getFileSize(), "myMessage", Integer.valueOf(42))
                    ))
            ),
            error, now, now, segmentName)
        );
    }
} catch (final SQLException exc)
{
    final String msg = "Error handling DB connection";
    throw new SdiException("DbError2", msg, exc, msg);
} catch (final MleOutboundException exc)
{
    throw new SdiException(exc.getLanguageKey(), exc.getShortMessage(), exc, exc.getParameters());
}

private static void callServices(final List<RecordData> recordDataList) throws IOException
{
    final ObjectMapper mapper = new ObjectMapper();
    final CookieManager cookieManager = new CookieManager();
    final String baseUrl = "https://localhost:8080/data/";
    final String orderUrl = baseUrl + "BOOrder/insert";
    final String operationUrl = baseUrl + "BOOperation/insert";
    for (final RecordData recordData : recordDataList)
    {
        final String orderJson = "{" +
            unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
                new Param("order.id", extract(recordData.getSapSdata(), 0, 40)),
                new Param("order.ordertype", extract(recordData.getSapSdata(), 40, 5)),
                new Param("order.article", extract(recordData.getSapSdata(), 45, 40))
            )))) +
            ", \"columns\" : []}";
        final String operationJson = "{" +
            unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
                new Param("operation.id", extract(recordData.getSapSdata(), 0, 40) + extract(recordData.getSapSdata(), 85,
4)),
                new Param("operation.designation", extract(recordData.getSapSdata(), 89, 40)),
                new Param("operation.act.scheduled", extract(recordData.getSapSdata(), 129, 1)),
                new Param("operation.plan.workplace", extract(recordData.getSapSdata(), 130, 8)),
                new Param("operation.processing_code", extract(recordData.getSapSdata(), 138, 6))
            )))) +
            ", \"columns\" : []}";

        new CommonHttpClient()
            .withCookieSupport(cookieManager)
            .withBasicAuthData("12345", "mpdv")
            .withRequestSettings(urlCon -> {
                urlCon.setRequestMethod("POST");
                urlCon.setRequestProperty("Content-Type", "application/json");
            })
            .communicate(orderUrl, orderJson);

        new CommonHttpClient()
            .withCookieSupport(cookieManager)
            .withBasicAuthData("12345", "mpdv")
            .withRequestSettings(urlCon -> {
                urlCon.setRequestMethod("POST");
                urlCon.setRequestProperty("Content-Type", "application/json");
            })
            .communicate(operationUrl, operationJson);
    }
}

private static ToIntBiFunction<CharSequence, String> writeFile(final ISystemUtilFactory factory)
{
    return (content, fileName) -> {
        final IPathManagementOpenFileForWritingAction writeAction =
factory.fetchUtil("PathManagementOpenFileForWritingAction");
        try (final OutputStream out = writeAction.writeFile("MOCLOGS", fileName))
        {
            final byte[] data = content.toString().getBytes(StandardCharsets.UTF_8);
            out.write(data);
            out.flush();
        }
    };
}

```

```

        return data.length;
    } catch (final RuntimeException | IOException ignore)
    {
    }
    return -1;
}

@JsonIgnoreProperties(ignoreUnknown = true)
static class Dummy
{
    private final List<Param> params;

    Dummy(final List<Param> params)
    {
        this.params = params;
    }

    public List<Param> getParams()
    {
        return this.params;
    }
}

@JsonIgnoreProperties(ignoreUnknown = true)
static class Param
{
    private final String acronym;
    private final String value;

    Param(final String acronym,
          final String value)
    {
        this.acronym = acronym;
        this.value = value;
    }

    public String getAcronym()
    {
        return this.acronym;
    }

    public String getValue()
    {
        return this.value;
    }
}
}

```

1.4 Code example hierarchical output data

This code example shows an ExternalJavaService that processes hierarchical MLE output data. The service reads all segments of type "RESTOUTBOUND1LEVELPARENT" and their sub segments. Then the service uses the order and the operation data from the MLE output data to create orders and operations via WSP and REST call.

```

package de.mpdv.extSvc.mleoutboundprocessor;

import static de.mpdv.commonmleoutboundutil.MleOutboundUtil.*;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.databind.ObjectMapper;
import de.mpdv.commonmleoutboundutil.BeginOutboundTaPacketParam;
import de.mpdv.commonmleoutboundutil.BeginOutboundTaParam;
import de.mpdv.commonmleoutboundutil.ControlRecordData;
import de.mpdv.commonmleoutboundutil.EndOutboundTaPacketParam;
import de.mpdv.commonmleoutboundutil.MleFileData;
import de.mpdv.commonmleoutboundutil.MleOutboundException;
import de.mpdv.commonmleoutboundutil.ProcessingState;
import de.mpdv.commonmleoutboundutil.ProtocolFileData;
import de.mpdv.commonmleoutboundutil.RecordData;
import de.mpdv.commonwebutil.CommonHttpClient;
import de.mpdv.sdi.data.SdiException;
import de.mpdv.sdi.data.SesContext;
import de.mpdv.sdi.data.SesRequest;
import de.mpdv.sdi.data.SesResult;
import de.mpdv.sdi.simpleExternalService.ISimpleExternalService;
import de.mpdv.sdi.systemutility.IDbConnectionProvider;
import de.mpdv.sdi.systemutility.IPathManagementOpenFileForWritingAction;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.io.IOException;
import java.io.OutputStream;
import java.net.CookieManager;
import java.nio.charset.StandardCharsets;
import java.sql.Connection;
import java.sql.SQLException;

```

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Calendar;
import java.util.Collections;
import java.util.List;
import java.util.function.ToIntBiFunction;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class MleOutboundHierarchicalProcessorService implements ISimpleExternalService
{
    @Override
    public SesResult execute(final SesRequest request,
        final SesContext context,
        final ISystemUtilFactory factory)
    {
        final String segmentName = "RESTOUTBOUND1LEVELPARENT";

        final Calendar now = (Calendar) context.getHydraNow().clone();
        final IDbConnectionProvider dbConnectionProvider = factory.fetchUtil("DbConnectionProvider");
        try (final Connection con = dbConnectionProvider.fetchDbConnection())
        {
            final String processIdentifier = createUniqueIdentifier();
            final String taId = createUniqueIdentifier();

            final List<RecordData> parentRecordList = beginOutboundTa(new BeginOutboundTaParam(con, processIdentifier, now,
segmentName));
            final CookieManager cookieManager = new CookieManager();
            final ObjectMapper mapper = new ObjectMapper();
            for (final RecordData parentRecord : parentRecordList)
            {
                ProcessingState state = ProcessingState.DONE;
                Throwable error = null;

                final List<RecordData> childRecordList = beginOutboundTaPacket(
                    new BeginOutboundTaPacketParam(con, processIdentifier, parentRecord.getInternalId(), now));
                final List<Integer> successfulRecords = new ArrayList<>();
                try
                {
                    callParentService(mapper, cookieManager, parentRecord);
                    successfulRecords.add(Integer.valueOf(parentRecord.getInternalId()));
                    for (final RecordData childRecord : childRecordList)
                    {
                        callChildService(mapper, cookieManager, parentRecord, childRecord);
                        successfulRecords.add(Integer.valueOf(childRecord.getInternalId()));
                    }
                } catch (final RuntimeException | Error exc)
                {
                    error = exc;
                    state = ProcessingState.ERROR;
                    throw exc;
                } catch (final IOException exc)
                {
                    error = exc;
                    state = ProcessingState.ERROR;
                    final String msg = "Error calling services";
                    throw new SdiException("lkinvalidState", msg, exc, msg);
                } finally
                {
                    final MleFileData protFileData = writeProtocolFile(writeFile(factory), "Protocol data", now);
                    final MleFileData errFileData = writeErrorFile(writeFile(factory), "Error data", now);
                    final MleFileData dataFileData = writeDataFile(writeFile(factory), "Data", now);

                    endOutboundTaPacket(
                        new EndOutboundTaPacketParam(con, processIdentifier, parentRecord.getInternalId(),
Collections.singletonList(
                            new ControlRecordData(taId, state).withProtocolFileDataList(Collections.singletonList(
                                new ProtocolFileData("Test", "TstPrg", protFileData.getFileName(), protFileData.getFileSize(),
errFileData.getFileName(), errFileData.getFileSize(), dataFileData.getFileName(),
dataFileData.getFileSize(),
"myMessage", Integer.valueOf(43))
                            )
                        ), error, now, now, segmentName)
                    .withSuccessfulRecords(createListOfAllRecords(parentRecord, childRecordList), successfulRecords)
                );
            }
        }
        return null;
    } catch (final SQLException exc)
    {
        final String msg = "Error handling DB connection";
        throw new SdiException("lkdError2", msg, exc, msg);
    } catch (final MleOutboundException exc)
    {
        throw new SdiException(exc.getLanguageKey(), exc.getShortMessage(), exc, exc.getParameters());
    }
}

private static List<Integer> createListOfAllRecords(final RecordData parentRecord,
    final List<RecordData> childRecordList)
{
    return Stream.concat(Stream.of(parentRecord), childRecordList.stream())
        .map(record -> Integer.valueOf(record.getInternalId()))
        .collect(Collectors.toList());
}

private static void callParentService(final ObjectMapper mapper,
    final CookieManager cookieManager,

```

```

        final RecordData parentRecord) throws IOException
    {
        final String orderUrl = "https://localhost:8080/data/BOOrder/insert";
        final String orderJson = "{" +
            unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
                new Param("order.id", extract(parentRecord.getSapSdata(), 0, 40)),
                new Param("order.ordertype", extract(parentRecord.getSapSdata(), 40, 5)),
                new Param("order.article", extract(parentRecord.getSapSdata(), 45, 40))
            )))) +
            ", \"columns\" : []";

        new CommonHttpClient()
            .withCookieSupport(cookieManager)
            .withBasicAuthData("12345", "mpdv")
            .withRequestSettings(urlCon -> {
                urlCon.setRequestMethod("POST");
                urlCon.setRequestProperty("Content-Type", "application/json");
            })
            .communicate(orderUrl, orderJson);
    }

    private static void callChildService(final ObjectMapper mapper,
        final CookieManager cookieManager,
        final RecordData parentRecord,
        final RecordData childRecord) throws IOException
    {
        final String operationUrl = "https://localhost:8080/data/BOOperation/insert";
        final String operationJson = "{" +
            unwrapJson(mapper.writeValueAsString(new Dummy(Arrays.asList(
                new Param("operation.id", extract(parentRecord.getSapSdata(), 0, 40) + extract(childRecord.getSapSdata(), 0, 4)),
                new Param("operation.designation", extract(childRecord.getSapSdata(), 4, 40)),
                new Param("operation.act.scheduled", extract(childRecord.getSapSdata(), 44, 1)),
                new Param("operation.plan.workplace", extract(childRecord.getSapSdata(), 45, 8)),
                new Param("operation.processing_code", extract(childRecord.getSapSdata(), 53, 6))
            )))) +
            ", \"columns\" : []";

        new CommonHttpClient()
            .withCookieSupport(cookieManager)
            .withBasicAuthData("12345", "mpdv")
            .withRequestSettings(urlCon -> {
                urlCon.setRequestMethod("POST");
                urlCon.setRequestProperty("Content-Type", "application/json");
            })
            .communicate(operationUrl, operationJson);
    }

    private static ToIntBiFunction<CharSequence, String> writeFile(final ISystemUtilFactory factory)
    {
        return (content, fileName) -> {
            final IPathManagementOpenFileForWritingAction writeAction =
factory.fetchUtil("PathManagementOpenFileForWritingAction");
            try (final OutputStream out = writeAction.writeFile("MOCLOGS", fileName))
            {
                final byte[] data = content.toString().getBytes(StandardCharsets.UTF_8);
                out.write(data);
                out.flush();
                return data.length;
            } catch (final RuntimeException | IOException ignore)
            {
                return -1;
            }
        };
    }

    @JsonIgnoreProperties(ignoreUnknown = true)
    static class Dummy
    {
        private final List<Param> params;

        Dummy(final List<Param> params)
        {
            this.params = params;
        }

        public List<Param> getParams()
        {
            return this.params;
        }
    }

    @JsonIgnoreProperties(ignoreUnknown = true)
    static class Param
    {
        private final String acronym;
        private final String value;

        Param(final String acronym,
            final String value)
        {
            this.acronym = acronym;
            this.value = value;
        }

        public String getAcronym()
        {
            return this.acronym;
        }

        public String getValue()
        {

```

```
        return this.value;
    }
}
```

1.5 API reference



You cannot use classes and functions that are not documented when you use the MDS license.



The API reference below is only available in English.

1.5.1 MleOutboundUtil

1.5.1.1 createUniqueIdentifier(): String

Creates an UUID encoded as Base64 URL encoded string of size 22 characters. Useful as processing identifier and TA-IDs.

Return:

Base64 URL encoded UUID

1.5.1.2 extract(String sapSdata, int start, int length): String

Extracts and trims a substring from sapSdata

Input:

sapSdata: data record string

start: start position

length: length

Return:

extracted string

NullPointerException: if *sapSdata* is NULL

1.5.1.3 unwrapJson(String json): String

Removes the curly bracket from start and end of JSON string to create a JSON snippet that is embeddable

Input:

json: JSON string

Return:

JSON string without first and last character

NullPointerException: if *json* is NULL

1.5.1.4 beginOutboundTa(BeginOutboundTaParam param): List<RecordData>

Hierarchical and flat records: Starts the processing of all records of a given segment name and returns the list of open records matching the given segment name. Marks all records with the processing identifier

Input:

param: parameter structure

Return:

list of outbound records matching the provided segment name. Empty list, if no open records exist

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

1.5.1.5 endOutboundTa(EndOutboundTaParam param): void

Only flat records: Ends the processing of one or all records marked with the provided processing identifier.

Input:

param: parameter structure

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

1.5.1.6 beginOutboundTaPacket(BeginOutboundTaPacketParam param): List<RecordData>

Only hierarchical records: Begins a processing of all children of a provided parent internal ID

Input:

param: parameter structure

Return:

child records

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

1.5.1.7 endOutboundTaPacket(EndOutboundTaPacketParam param): void

Only hierarchical records: Ends the processing of all children

Input:

param: parameter structure

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

1.5.1.8 fetchOutboundConfigStructure(FetchOutboundConfigStructureParam param): Optional<MleOutboundConfigStruct>

Fetches MLE outbound configuration data from distribution model, logical system and logical system configuration

Input:

param: parameter structure

Return:

MLE outbound configuration data or empty structure if no or only inactive logical systems exist

Throws:

MleOutboundException: if an error occurs

NullPointerException: if *param* is NULL

1.5.1.9 writeProtocolFile(ToIntBiFunction<CharSequence, String> writeFunction, CharSequence content, Calendar now): MleFileData

Writes a protocol file by creating a file name matching following pattern: "PRO-yyyy-MM-dd HH:mm:ss.SSS\d\d\d\d\d.txt". The pattern of 5 times \d means a five digit random number.

Input:

writeFunction: function to write the file

content: file content

now: timestamp used for file name

Return:

data structure containing file name and file size

1.5.1.10 writeErrorFile(ToIntBiFunction<CharSequence, String> writeFunction, CharSequence content, Calendar now): MleFileData

Writes an error file by creating a file name matching following pattern: "ERR-yyyy-MM-dd HH:mm:ss.SSS\d\d\d\d\d.txt". The pattern of 5 times \d means a five digit random number.

Input:

writeFunction: function to write the file

content: file content

now: timestamp used for file name

Return:

data structure containing file name and file size

1.5.1.11 writeDataFile(ToIntBiFunction<CharSequence, String> writeFileFunction, CharSequence content, Calendar now): MleFileData

Writes a data file by creating a file name matching following pattern: "DAT-yyyy-MM-dd HH:mm:ss.SSS\d\d\d\d\d.txt". The pattern of 5 times \d means a five digit random number.

Input:

writeFileFunction: function to write the file

content: file content

now: timestamp used for file name

Return:

data structure containing file name and file size

1.5.1.12 resetAllInProcessDataRecords(ResetAllInProcessData RecordsParam param): void

Hierarchical and flat records: Resets all data records to status TODO (001) that are marked with the provided processing identifier.

Input:

parameter structure

1.5.1.13 resetInProcessDataRecordRecursive(ResetInProcessD ataRecordRecursiveParam param): void

Hierarchical and flat records: Resets provided data record and their children to status TODO (001) that are marked with the provided processing identifier.

Input:

parameter structure

1.5.2 MleOutboundException

This exception carries the error as a translatable key and corresponding parameters. Additionally this exception contains an english short message.

1.5.2.1 **getLanguageKey(): String**

Gets the language key that is used for translation. The language key is a string that is used to lookup a translated text.

Return:

language key that is used to lookup translated text

1.5.2.2 **getShortMessage(): String**

Gets the English error message that is meant for logging

Return:

English error message

1.5.2.3 **getParameterList(): List<String>**

Gets the parameters for the translation text as List

Return:

parameter list

1.5.2.4 **getParameters(): List<String>**

Gets the parameters for the language key as array.

Return:

parameter array

1.5.3 MleFileData

This data structure contains a file name and a file size

1.5.3.1 **getFileName(): String**

Gets the file name without path

Return:

file name without path

1.5.3.2 **getFileSize(): int**

Gets the file size

Return:

file size

1.5.4 RecordData

This data structure contains the data of a MLE outbound data record and is immutable.

**1.5.4.1 RecordData(String dbTald,
 String recordStatus,
 Calendar recordSaveTs,
 Calendar recordWorkTs,
 String recordSourceSystem,
 String sapSegnam,
 String sapMandt,
 String sapDocnum,
 String sapSegnum,
 String sapPsgnum,
 String sapHlevel,
 String sapSdata,
 String pid,
 String param2,
 int internalId,
 Integer parentInternalId)**

Constructor

1.5.5 BeginOutboundTaParam

This data structure contains all parameters for MleOutboundUtil.beginOutboundTa() and is immutable..

1.5.5.1 BeginOutboundTaParam(Connection con, String processingIdentifier, Calendar workTs, String segmentName)

Constructor

Input:

con: DB connection

processingIdentifier: marker for the current processing thread: unique identifier

workTs: the work timestamp to use for the records

segmentName: segment name

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any non-string value is NULL

1.5.5.2 withPropagateTransactionMode()

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

1.5.6 EndOutboundTaParam

This data structure contains all parameters for MleOutboundUtil.endOutboundTa() and is immutable.

1.5.6.1 EndOutboundTaParam(Connection con, String processingIdentifier, List<ControlRecordData> controlRecordDataList, Throwable exceptionOfTryBlock, Calendar workTs, Calendar saveTs, String segmentName)

Constructor: The values of IDoc type, CIM type, message type, message code and message function are taken from the configuration of the segment.

Input:

con: DB Connection

processingIdentifier: unique identifier that was used to mark the records

controlRecordDataList: list of of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

**1.5.6.2 EndOutboundTaParam(Connection con,
String processingIdentifier,
List<ControlRecordData> controlRecordDataList,
Throwable exceptionOfTryBlock,
Calendar workTs,
Calendar saveTs,
String segmentName,
String dmSapIdotyp,
String dmSapCimtyp,
String dmSapMesTyp,
String dmSapMescod,
String dmSapMesfct)**

Constructor

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

controlRecordDataList: list of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

dmSapIdotyp: IDoc type

dmSapCimtyp: CIM type

dmSapMesTyp: message type

dmSapMescod: message code

dmSapMesfct: message function

1.5.6.3 withSuccessfulRecords(List<Integer> allRecords, List<Integer> successfulRecords): EndOutboundTaParam

Sets the state of successful records to DONE while all other records get the state of the control record.

The invocation of this method is only possible if there is only one control record.

Input:

allRecords: all internal IDs of all records

successfulRecords: internal IDs of successful records only

Return:

new instance with applied settings

Throws:

NullPointerException: if any of the parameters is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

1.5.6.4 withDataRecordClassifierList(List<DataRecordClassifier> dataRecordClassifierList): EndOutboundTaParam

Sets individual state codes to each data record

Input:

dataRecordClassifierList list of data record internal ID to state

Return:

new instance with applied settings

Throws:

NullPointerException: if the parameter is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

1.5.6.5 withPropagateTransactionMode(): EndOutboundTaParam

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

1.5.7 BeginOutboundTaPacketParam

This data structure contains all parameters for `MleOutboundUtil.beginOutboundTaPacket()` and is immutable.

1.5.7.1 **BeginOutboundTaPacketParam(Connection con, String processingIdentifier, int parentInternalId, Calendar workTs)**

Constructor

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

parentInternalId: internal ID of parent record

workTs: work timestamp

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

1.5.7.2 **withPropagateTransactionMode(): BeginOutboundTaPacketParam**

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

1.5.8 EndOutboundTaPacketParam

This data structure contains all parameters for MleOutboundUtil.endOutboundTaPacket() and is immutable.

1.5.8.1 EndOutboundTaPacketParam(Connection con, String processingIdentifier, int parentInternalId, List<ControlRecordData> controlRecordDataList, Throwable exceptionOfTryBlock, Calendar workTs, Calendar saveTs, String segmentName)

Constructor: The values of IDoc type, CIM type, message type, message code and message function are taken from the configuration of the segment..

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

parentInternalId: internal ID of parent record

controlRecordDataList: list of of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

**1.5.8.2 EndOutboundTaPacketParam(Connection con,
String processingIdentifier,
int parentInternalId,
List<ControlRecordData> controlRecordDataList,
Throwable exceptionOfTryBlock,
Calendar workTs,
Calendar saveTs,
String segmentName,
String dmSapIdotyp,
String dmSapCimtyp,
String dmSapMesTyp,
String dmSapMescod,
String dmSapMesfct)**

Constructor

Input:

con: DB connection

processingIdentifier: unique identifier that was used to mark the records

parentInternalId: internal ID of parent record

controlRecordDataList: list of of data necessary for control record

exceptionOfTryBlock: exception of the try block. If the exception of the try block is not passed in then any exception of this method hides the exception of the try block

workTs: work timestamp

saveTs: save timestamp

segmentName: segment name

dmSapIdotyp: IDoc type

dmSapCimtyp: CIM type

dmSapMesTyp: message type

dmSapMescod: message code

dmSapMesfct: message function

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

1.5.8.3 withSuccessfulRecords(List<Integer> allRecords, List<Integer> successfulRecords): EndOutboundTaPacketParam

Sets the state of successful records to DONE while all other records get the state of the control record.
The invocation of this method is only possible if there is only one control record.

Input:

allRecords: all internal IDs of all records

successfulRecords: internal IDs of successful records only

Return:

new instance with applied settings

Throws:

NullPointerException: if any parameter is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

1.5.8.4 withDataRecordClassifierList(List<DataRecordClassifier> dataRecordClassifierList): EndOutboundTaPacketParam

Sets individual states to each data record

Input:

dataRecordClassifierList: list of data record internal ID to state

Return:

new instance with applied settings

Throws:

NullPointerException: if parameter is NULL

IllegalStateException: if the invocation of this method is not allowed due to other method calls

1.5.8.5 withPropagateTransactionMode(): EndOutboundTaPacketParam

Enables the use of the auto commit mode from caller. This can be used to participate in another transaction.

Return:

new instance with applied settings

1.5.9 ControlRecordData

This data structure contains data for MLE outbound control records and is immutable. This data structure contains optionally also a list of file records.

1.5.9.1 **ControlRecordData(String tald, ProcessingState state)**

Constructor

Input:

tald: transaction ID of the control record

state: processing code of the control record

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

1.5.9.2 **withProtocolFileDataList(List<ProtocolFileData> protocolDataList): ControlRecordData**

Adds file records to the outbound control record.

Input:

protocolDataList: list of file records. Each file record may contain a protocol file, an error file and a data file

Return:

new instance with applied settings

Throws:

NullPointerException: if the parameter is NULL

1.5.10 **ProtocolFileData**

This data structure contains data for the file record of outbound MLE. This data structure is immutable.

1.5.10.1 ProtocolFileData(String designation, String program, String protocolFileName, int protocolFileSize, String errorFileName, int errorFileSize, String dataFileName, int dataFileSize, String message, Integer textNo)

Constructor

Input:

designation: mandatory description of the file record

program: optional program name

protocolFileName: optional file name of the protocol file

protocolFileSize: file size of the protocol file or 0 if no protocol file is provided

errorFileName: optional file name of the error file

errorFileSize: file size of the protocol file or 0 if no error file is provided

dataFileName: optional file name of the data file

dataFileSize: file size of the protocol file or 0 if no data file is provided

message: optional message assigned to the file record

textNo: optional text number assigned to the file record

Throws:

IllegalArgumentException: if the designation is NULL or empty

1.5.11 DataRecordClassifier

This data structure allows to set individual state codes at each data record. This data structure is immutable.

1.5.11.1 DataRecordClassifier(String talId, ProcessingState dataState, Integer internalId): ControlRecordData

Constructor

Input:

talId: transaction ID of the data record

dataState: processing code of the data record

internalId: internal ID of the data record

Throws:

IllegalArgumentException: if talId is NULL or empty

NullPointerException: if dataState or internalId is NULL

1.5.12 ProcessingState

Enumeration of processing codes for MLE outbound

1.5.12.1 getControlRecordCode(): String

Gets the processing code for a control record

1.5.12.2 getDataRecordCode(): String

Gets the processing code for a data record

1.5.13 FetchOutboundConfigStructureParam

This data structure contains all parameters for `MleOutboundUtil.fetchOutboundParameterStructure()` and is immutable.

1.5.13.1 FetchOutboundConfigStructureParam(Connection con, String segmentName, boolean isEncryptedPassword)

Constructor

Input:

con: DB connection

segmentName: segment name

isEncryptedPassword: is the password in the DB encrypted

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

1.5.14 MleOutboundConfigStruct

This data structure contains all data from the MLE outbound configuration and is immutable.

1.5.14.1 getDmDirect(): String

Getter

Return:

direction: I for input; O for output

1.5.14.2 getDmDesc(): String

Getter

Return:

description

1.5.14.3 getDmSapMestyp(): String

Getter

Return:

message type

1.5.14.4 getDmSapIdoctyp(): String

Getter

Return:

IDoc type

1.5.14.5 getDmSapCimtyp(): String

Getter

Return:

CIM type

1.5.14.6 getDmSapMescod(): String

Getter

Return:

message code

1.5.14.7 getDmSapMesfct(): String

Getter

Return:

message function

1.5.14.8 getDmSourcesys(): String

Getter

Return:

SAP target system

1.5.14.9 getDmSapMescod(): String

Getter

Return:

message code

1.5.14.10 getDmSapSegnam01(): String

Getter

Return:

segment name for upload

1.5.14.11 getDmSapSegnam02(): String

Getter

Return:

segment name 2

1.5.14.12 getDmSapSegnam03(): String

Getter

Return:

segment name 3

1.5.14.13 getDmSapSegnam04(): String

Getter

Return:

segment name 4

1.5.14.14 getDmSapSegnam05(): String

Getter

Return:

segment name 5

1.5.14.15 getDmSapSegnam06(): String

Getter

Return:

segment name 6

1.5.14.16 getDmSapSegnam07(): String

Getter

Return:

segment name 7

1.5.14.17 getDmSapSegnam08(): String

Getter

Return:

segment name 8

1.5.14.18 getDmSapSegnam09(): String

Getter

Return:

segment name 9

1.5.14.19 getDmSapSegnam10(): String

Getter

Return:

segment name 10

1.5.14.20 getDmSapTest(): String

Getter

Return:

test flag

1.5.14.21 getDmDest(): String

Getter

Return:

logical target system for output process

1.5.14.22 getDmParam1(): String

Getter

Return:

value of parameter 1

1.5.14.23 getDmParam2(): String

Getter

Return:

value of parameter 2

1.5.14.24 getLsLogsys(): String

Getter

Return:

logical system

1.5.14.25 getLsDesc(): String

Getter

Return:

description of logical system

1.5.14.26 getLsParam1(): String

Getter

Return:

communication mode

1.5.14.27 getLsParam2(): String

Getter

Return:

value of parameter 2

1.5.14.28 getLsActRole(): String

Getter

Return:

active role

1.5.14.29 getLscParamName01(): String

Getter

Return:

parameter name 1

1.5.14.30 getLscParamVal01(): String

Getter

Return:

parameter value 1

1.5.14.31 getLscParamName02(): String

Getter

Return:

parameter name 2

1.5.14.32 getLscParamVal02(): String

Getter

Return:

parameter value 2

1.5.14.33 getLscParamName03(): String

Getter

Return:

parameter name 3

1.5.14.34 getLscParamVal03(): String

Getter

Return:

parameter value 3

1.5.14.35 getLscParamName04(): String

Getter

Return:

parameter name 4

1.5.14.36 getLscParamVal04(): String

Getter

Return:

parameter value 4

1.5.14.37 getLscParamName05(): String

Getter

Return:

parameter name 5

1.5.14.38 getLscParamVal05(): String

Getter

Return:

parameter value 5

1.5.14.39 getLscParamName06(): String

Getter

Return:

parameter name 6

1.5.14.40 getLscParamVal06(): String

Getter

Return:

parameter value 6

1.5.14.41 getLscParamName07(): String

Getter

Return:

parameter name 7

1.5.14.42 getLscParamVal07(): String

Getter

Return:

parameter value 7

1.5.14.43 getLscParamName08(): String

Getter

Return:

parameter name 8

1.5.14.44 getLscParamVal08(): String

Getter

Return:

parameter value 8

1.5.14.45 getLscParamName09(): String

Getter

Return:

parameter name 9

1.5.14.46 getLscParamVal09(): String

Getter

Return:

parameter value 9

1.5.14.47 getLscParamName10(): String

Getter

Return:

parameter name 10

1.5.14.48 getLscParamVal10(): String

Getter

Return:

parameter value 10

1.5.14.49 getLscParamName11(): String

Getter

Return:

parameter name 11

1.5.14.50 getLscParamVal11(): String

Getter

Return:

parameter value 11

1.5.14.51 getLscParamName12(): String

Getter

Return:

parameter name 12

1.5.14.52 getLscParamVal12(): String

Getter

Return:

parameter value 12

1.5.14.53 getLscParamName13(): String

Getter

Return:

parameter name 13

1.5.14.54 getLscParamVal13(): String

Getter

Return:

parameter value 13

1.5.14.55 getLscParamName14(): String

Getter

Return:

parameter name 14

1.5.14.56 getLscParamVal14(): String

Getter

Return:

parameter value 14

1.5.14.57 getLscParamName15(): String

Getter

Return:

parameter name 15

1.5.14.58 getLscParamVal15(): String

Getter

Return:

parameter value 15

1.5.14.59 getLscSapSndPor(): String

Getter

Return:

sending system port

1.5.14.60 getLscSapSndPrt(): String

Getter

Return:

partner type of sending system

1.5.14.61 getLscSapSndPrn(): String

Getter

Return:

partner number of sender

1.5.14.62 getLscSapRcvPor(): String

Getter

Return:

receiver port

1.5.14.63 getLscSapRcvPrt(): String

Getter

Return:

partner type of receiver

1.5.14.64 getLscSapRcvPrn(): String

Getter

Return:

partner number of receiver

1.5.14.65 getLscParam1(): String

Getter

Return:

value of parameter 1

1.5.14.66 getLscParam2(): String

Getter

Return:

value of parameter 2

1.5.14.67 getLscProgTyp(): String

Getter

Return:

program type either ?C for client or ?S for server

1.5.15 ResetAllInProcessDataRecordsParam

This data structure contains all parameters for MleOutboundUtil.resetAllInProcessDataRecords() and is immutable.

1.5.15.1 ResetAllInProcessDataRecordsParam(Connection con, String processingIdentifier)

Constructor

Input:

con: DB Connection

processingIdentifier: unique identifier that was used to mark the records

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL

1.5.16 ResetInProcessDataRecordRecursiveParam

This data structure contains all parameters for MleOutboundUtil.resetInProcessDataRecordRecursive() and is immutable.

1.5.16.1 ResetInProcessDataRecordRecursiveParam(Connection con, String processingIdentifier, int internalId)

Constructor

Input:

con: DB Connection

processingIdentifier: unique identifier that was used to mark the records

internalId: internal ID of record to reset

Throws:

IllegalArgumentException: if any mandatory string value is NULL or empty

NullPointerException: if any mandatory non-string value is NULL